Project 3

Advanced Compilers

Released: May 7, 2026
Due: May 28, 2026

By using Static Single Assignment (SSA) Intermediate Representation (IR), the well-known Sparse Conditional Constant Propagation (SCCP) algorithm provides fast and powerful transformations [1]. Many modern compilers, including LLVM[1], implement and use SCCP widely. In this project, we will implement simple SCCP analysis for integer constants as an LLVM pass plugin.

1 Background

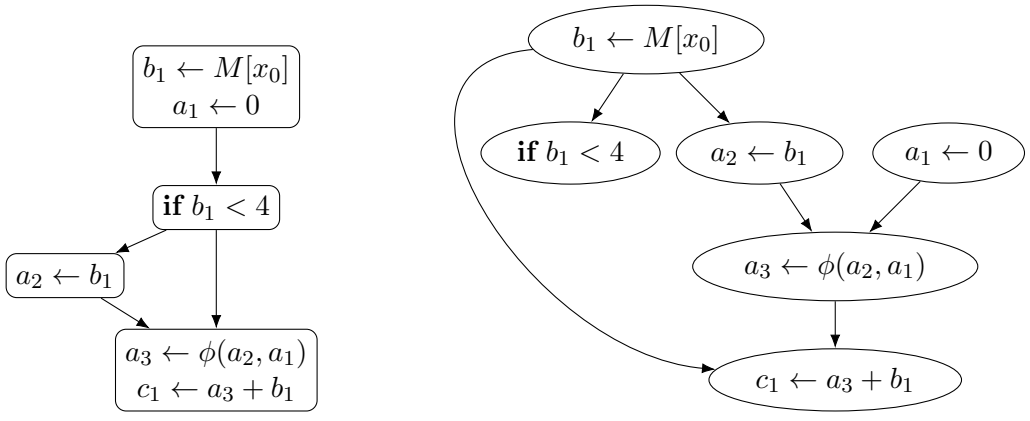
1.1 SSA

1.1.1 Dominating relationship

In an SSA-formed language, each value of a variable is statically distinguishable. Thus, *Use*s of a value are correlated with a unique *Def* of a value. In other words, in SSA, all *Use*s of a variable are dominated by the variable's *Def*.

1.1.2 SSA graph

Due to the dominating relationships in SSA, *Def-Use* and *Use-Def* chains are condensed. Since the corresponding definition of a variable is unique to its *Use*s, one can build a graph based on variables' *Use-Def* chains called an SSA graph.



(a) CFG (b) SSA Graph

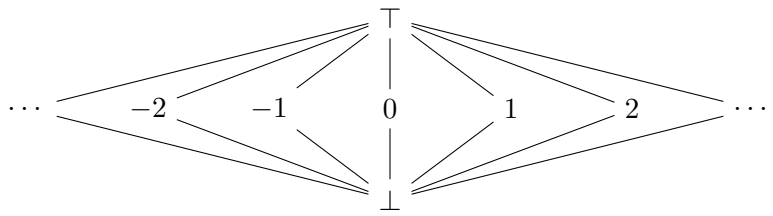
Figure 1: CFG and corresponding SSA graph.

[1]LLVM's implementation additionally uses the dominator tree to manage unreachable blocks.

2 Sparse Conditional Constant Propagation [1]

Just like traditional dataflow analyses, the SCCP algorithm traverses the Control Flow Graph (CFG). However, unlike traditional analyses, SCCP also disseminates dataflow information through SSA graph edges. Thanks to the sparsity of the SSA graph and the dominating relationships inherent in SSA form, SCCP can propagate information efficiently. Also, SCCP continuously evaluates branch conditions and executable paths using the information available so far. Thus, SCCP can determine whether a variable is constant depending on branches with constant conditions.

2.1 Lattice



$$\begin{aligned} \top &: \text{Undefined} \\ C := \{\ldots, -1, 0, 1, \ldots\} &: \text{Integer constant} \\ \bot &: \text{Not a constant} \end{aligned}$$

2.1.1 Meet[2]

$$\begin{aligned} x \sqcap \top &= x \\ x \sqcap \bot &= \bot \\ c_1 \sqcap c_2 &= c_1 \quad (c_1 = c_2) \\ c_1 \sqcap c_2 &= \bot \quad (c_1 \neq c_2) \end{aligned}$$

[2]Since '$\wedge$' and '$\vee$' can be confused with logical AND and OR, '$\sqcap$' and '$\sqcup$' are used to denote meet and join, respectively.

2.2 Algorithm[2, p. 101]

Algorithm 1 Sparse Conditional Constant Propagation

```
Input: CFG, SSA graph
CFGWorklist ← {Edge(NULL, entry)}
SSAWorklist ← ∅
while (!CFGWorklist.Empty || !SSAWorklist.Empty) do
    x ←pop one from any list
    if x is from CFGWorklist then
        E ← x
        B ←destination(E)
        markExecutable(E)
        visit(φ)    (∀φ ∈ B)
        if B.IsFirstVisit then
            visit(I)    (∀I ∈ B)
        if hasSingleSuccessor(B) then
            S ←getSingleSuccessor(B)
            E′ ←Edge(B, S)
            if !isExecutableEdge(E′) then
                CFGWorklist.append(E′)
    else /* x is from SSAWorklist */
        if x is a φ then
            visit(x)
        else
            if Any incoming CFG edge to x is executable then
                visit(x)
```

Algorithm 2 visit

```
Input: An instruction I
if I is a φ then
    Meet all the arguments from the executable edges.
else if I is a conditional branch then
    Evaluate the condition and append reachable edge to CFGWorklist.
else
    Update dataflow information using transfer function.

if Dataflow information of I has changed then
    Append all outgoing SSA edges of I to SSAWorklist.
```

3 Code

3.1 `LatticeValue`

Base class for lattice values. This class uses CRTP.

3.2 `SparseDataflowFramework`

Base class for analysis pass. In this project, our pass will be applied to functions.

3.3 `getId`

Returns unique id (name) of given `Value`. Note that some LLVM instructions may not have names (e.g., `br`, `ret`).

3.4 `ConstantValue`

Class for the lattice values of simple SCCP.

3.5 `CFGEdge`

Edge of CFG. `nullptr` is used to denote anonymous block outside the CFG. Some relational operators are also implemented to satisfy the ordering requirements of container classes.

3.6 `SimpleSCCP`

Analysis pass implementing simple SCCP for integer constants.

3.7 `SimpleSCCP::InstVisitor`

Instruction visitor used while implementing `visit`. This struct inherits LLVM's instruction visitor.

3.8 `SimpleSCCPPrinter`

Printer pass for `SimpleSCCP`. If you wish to print the analysis result of `SimpleSCCP` to a stream, use this helper pass.

Tips

- Please read LLVM Programmer's Manual and Doxygen References. Since the Doxygen references are generated from the latest version of LLVM, be **cautious** that there can be differences from the version we use (LLVM-19).
- You can find the LLVM source code at GitHub. Relevant implementation files include:
  - `llvm/lib/Transforms/Scalar/SCCP.cpp`
  - `llvm/lib/Transforms/Utils/SCCPSolver.cpp`
- Most containers' `operator[]` generates a default entry if there is no corresponding entry. Consider using `find` or other methods that do not automatically insert entries.
- `opt` prints out the IR through `stdout`. Information and statistics are usually printed to `stderr`.
- You should clean out the `build` directory before building other projects.

4 Input

Your analysis pass will receive IR after the `mem2reg` pass. The `mem2reg` pass promotes variables residing on the stack to virtual registers and prunes cumbersome SSA structures. Please refer to the Commands section for the commands to generate input IR.

5 Objectives

1. Implement all three `TODO`s in `SimpleSCCP.cpp`.
   - Your implementation should not change the given IR.
   - Printing order can be nondeterministic by the data structure (ADT) being used.
2. Write a brief review of your implementation.
3. Prepare your own `testcase.c` where SCCP can be applied. You may refer to `test.c` as an example.
4. Run your `SimpleSCCP` analysis on your testcase and include the analysis result in your report.

6 Commands

- Build your plugin.
  ```
  cmake -S . -G Ninja -B build
  ```
- Generate IR from the source.
  ```
  clang -O0 -S -emit-llvm -Xclang -disable-O0-optnone \
    -Xclang -disable-llvm-passes -fno-discard-value-names \
    test.c -o test.ll
  ```
- Run the `mem2reg` pass.
  ```
  opt -S -passes='mem2reg' test.ll -o input.ll
  ```
- Call your pass from `opt`.
  ```
  opt -S -load-pass-plugin build/lib/libSimpleSCCP.so \
    -passes='print<simple-sccp>' input.ll -disable-output 2> test.out
  ```

7 Example

```
1 %z.0 : { 5 }
2 %add : { 3 }
3 %cmp : { 0 }
4 %add1 : { 5 }
```

Figure 2: Output example of SimpleSCCP pass (test.out)

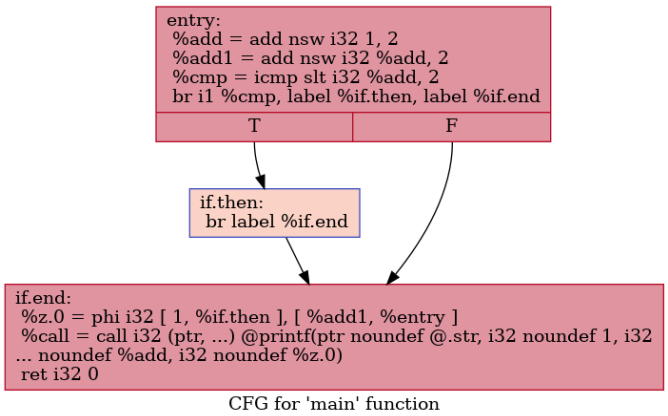


Figure 3: CFG of `input.ll`

8 Submission

- Submit a single zip archive to eTL.
- The archive must be in the `.zip` format. Other archive formats such as `.tar`, `.tar.gz`, `.tgz`, or `.7z` will not be accepted.
- The zip archive must contain exactly the following three files:
  - `SimpleSCCP.cpp`: your implementation.
  - `report.pdf`: a brief report that reviews your implementation and includes the analysis result for your own testcase. The report should discuss how your code aligns with the given pseudocode and show the `SimpleSCCP` analysis result for your testcase.
  - `testcase.c`: a source code example where SCCP can be applied.

References

[1] M. N. Wegman and F. K. Zadeck, "Constant propagation with conditional branches," *ACM Trans. Program. Lang. Syst.*, vol. 13, no. 2, pp. 181–210, Apr. 1991, ISSN: 0164-0925. DOI: 10.1145/103135.103136. [Online]. Available: https://doi.org/10.1145/103135.103136.

[2] F. Rastello and F. Tichadou, *SSA-based Compiler Design*. Springer International Publishing, 2022, pp. 95–106, ISBN: 9783030805142. [Online]. Available: https://books.google.co.kr/books?id=zS54zgEACAAJ.